

Technical System Specification

How the system works and how it must be structured. Normative for the builder agent: where the spec says **MUST**, the build **MUST** comply.

Version 1.0

Stack: Python 3.12 · PostgreSQL 15 + pgvector · \$0 infra

Companion: Dossier 2 — Builder Brief (how to build it)

evals/ cases.yaml run_evals.py

tests/ scripts/ migrations/

4 · Source plugin architecture (the modularity core)

4.1 Plugin contract

Every data endpoint is a subclass of SourcePlugin. The orchestrator only ever talks to this interface, which is what makes endpoints addable, removable and movable without touching pipeline code.

```
class SourcePlugin(ABC):  
  
    id: str          # e.g. "hn_firebase" (stable, used as DB key)  
  
    tier: Literal["S", "A"]  
  
    layers: list[str] # subset of ["L0", "L2"], from sources.yaml  
  
    budget_per_day: int; rps: float; cache_ttl_h: int  
  
    schedule: Literal["nightly", "weekly", "hourly", "on_demand"]
```

```
def fetch(self, ctx) -> RawBatch: ...
```

4.2 Registry (sources.yaml)

sources:

```
hn_firebase: { module: tier_s.hn,      tier: S, layers: [L0], schedule: nightly,  
budget_per_day: 2000, rps: 5 }  
  
wiki_pageviews:{ module: tier_s.wiki,   tier: S, layers: [L0], schedule: nightly,  
budget_per_day: 2000, rps: 4 }  
  
ebay_browse:  { module: tier_a.ebay,    tier: A, layers: [L2], schedule: on_demand,  
budget_per_day: 1000, rps: 1 }
```

4.3 Lifecycle rules

- **Add** — new module + registry entry + unit tests with recorded fixtures + 3 eval cases. No live calls in tests, ever.
- **Remove** — set enabled: false in the registry. Keep the module for 30 days before deleting.
- **Move layers** — edit only the layers list. The orchestrator MUST hard-fail at startup if a Tier-A source is listed in L0.
- **Budgets** — a central token bucket per source id. On HTTP 429 the plugin MUST back off and record quota_spend; the run continues without that source.

5 · Execution and state (never redo work)

5.1 Run ledger and watermarks

Every run writes a ledger row per (run, layer, source): status, item counts, quota spent, timestamps. Each source keeps a watermark (cursor or last-fetch timestamp). A rerun processes only what changed since the watermark.

5.2 Content-hash dedup

Fetches payloads are normalized then hashed (SHA-256). Identical hash to the stored one means skip parsing, skip scoring, skip LLM. This single mechanism is what keeps nightly runs near 10 minutes and LLM calls near 50.

5.3 Append-only snapshots

Scores are versioned rows (candidate, run_id, weights_version, five sub-scores, fad probability, revenue triple). History is never updated in place. Trends, backtests and weight learning all read the snapshot table.

5.4 Checkpoints, resume, TTL

- Checkpoint after every layer per candidate batch. A crashed L2 resumes unfinished candidates only.

- Raw lake rows expire after 90 days (TTL job). Snapshots and the ledger never expire.
- Determinism check: replaying a checkpoint with identical inputs MUST reproduce identical downstream scores (hash-asserted in CI).

Critical rule. Side effects (quota spend, external writes) MUST be recorded in a ledger table the checkpoint system does not own, applied with compare-and-swap. A resume MUST be physically unable to double-spend quota or double-emit. State that must survive a rollback lives outside the rollback surface.

6 · Scoring and LLM gates

6.1 Gap score (computed in code, never by the LLM)

$MGS = 0.30 \cdot DV + 0.25 \cdot (100 - SS) + 0.20 \cdot SP + 0.15 \cdot MP + 0.10 \cdot FE$. All inputs are per-category percentiles computed in SQL/Python. The fad flag (30d vs 90d vs 365d slope divergence, seasonal strength, evergreen ratio) is a classifier feature set, not a vibe. Revenue is Monte Carlo over TAM × CTR × CVR × price, reported as P10/P50/P90.

6.2 The three gates (only LLM call sites in v1)

Gate	Job	Budget
Planner (weekly)	User prompt -> search plan: keywords, sources, per-source budgets	~1 call
Judge (nightly)	Keep/drop + fad probability + enrich list, JSON, cited quotes	Batched, ~20 calls
Writer (nightly)	One-page brief: verdict, players, risks, angles, revenue reasoning	~30 calls, top-K only

6.3 Gateway, cache, schemas

- Provider chain with automatic failover: Gemini Flash -> Groq -> Cerebras -> local Ollama. Per-gate call and token caps; on cap, the gate degrades (skip with reason logged), never retries blindly.
- LLM cache keyed by hash of normalized input, 30-day TTL. Identical work never hits a provider twice.
- All gate outputs are Pydantic-validated JSON. Schema violation means retry once, then drop with reason.
- Grounding enforced in code: any factual claim without quote plus URL is stripped before it reaches scores or briefs.

7 · Data model (tables)

Table	Purpose (one line)
sources	Registry mirror: id, tier, layers, budgets, enabled, watermark
runs	Run ledger: id, started/finished, status per layer
run_source_log	Per (run, source): status, items, quota spent
raw_items	Lake: source, fetched_at, content_hash, payload ref. 90-day TTL
signals	Normalized facts: entity, source, metric, value, ts
candidates	Phrases/entities under evaluation with category
scores	Append-only snapshots: sub-scores, fad prob, revenue triple, weights version
briefs	Writer outputs linked to a score snapshot
quota_ledger	Side-effect ledger OUTSIDE checkpoints: spend records, compare-and-swap

llm_cache

Hash -> gate output, 30-day TTL

eval_cases

Golden cases with expected keep/drop and score bands

8 · Evals and CI (quality gate)

- 50 golden cases in evals/cases.yaml: input signals plus expected keep/drop, fad label, and score band.
- Single LLM-judge call per case scoring 0.0–1.0 plus pass/fail on: factual accuracy, citation accuracy, completeness, source quality, tool efficiency.
- Replay determinism: checkpoint replay MUST reproduce scores (hash match) or CI fails.
- Release rule: no rubric may drop more than 2 points from baseline on any PR. Evals run in CI before features merge.

9 · Observability and ops

- Structured logs with run_id on every line. Token accounting per gate per run.
- A health CLI (python -m trend_analyst.health) printing: last run status, per-source ok/degraded/down, quota burn vs budget, judge keep-rate, eval baseline. The monitor agent reads this, not raw logs.
- Alert thresholds: HTTP 429 rate, keep-rate drift beyond band, quota burn above 80 percent, eval drop. Alerts go to a local outbox file the monitor agent triages.

10 · Security and configuration

- pydantic-settings layered config: defaults < YAML < environment < CLI. Sample values only in .env.example.
- Secrets (API keys, tokens) live in environment or a local untracked file. NEVER in the repo, logs, snapshots, or eval fixtures. Tests use recorded fixtures with secrets redacted.
- Network egress allowlist derived from the registry: a source may only call its declared domains.
- The monitor agent runs with least privilege: read plus explicitly listed safe actions (full matrix in AGENT.md).

11 · Stack (pinned)

Choice	Version / note
Python	3.12, typed throughout (mypy-strict on new modules)
PostgreSQL + pgvector	15+, SQLAlchemy 2, migrations for every schema change
pydantic v2	Config, gate schemas, plugin manifests
httpx	All HTTP, with timeouts + retries + backoff everywhere
pytest + ruff	Tests mandatory per module; lint clean or no merge
GitHub Actions	Nightly cron + CI (tests, evals, determinism check)

Appendix A · Source catalog (ranked)

The full endpoint inventory, ranked by value per unit of quota for an indie operator. Tier S runs in L0 on schedule; Tier A runs in L2 on survivors only; dropped sources stay manual. Keep sources.yaml sorted in this order — the orchestrator iterates enabled sources in file order, so file order is execution order.

Tier S — L0 core, in pipeline order

#	Source	Role	Budget
---	--------	------	--------

1	HN Firebase	Tech launches + critiques	No key, no stated limit
2	Arctic Shift	Reddit pain mining, ~15 subs	No auth, polite (~2 req/s)
3	Bluesky API + Jetstream	Live social complaints	No auth, 3000 / 5 min
4	Wikipedia Pageviews	Absolute demand baseline	No key, generous
5	Google + Amazon Suggest	Long-tail intent expansion	No key, weekly, 1–2 s throttle
6	WordPress.org	Micro-SaaS gaps (installs vs rating)	No key, 1 req/s, weekly
7	Steam reviews	Game and tool sentiment gaps	No key, ~10 req/s
8	Shopify sitemap + products.json	DTC and app-store gaps	No key, 1 req/s, weekly
9	Mastodon trends	Social trend cross-check	No key, nightly
10	OpenAlex + arXiv	Paper-surge tech radar	mailto polite pool, weekly
11	GDELT DOC	News-cycle validation	No key, nightly
12	iTunes Search	App demand + ratings	No key, ~20/min, weekly
13	itch.io RSS	Pre-Steam experiments	No key, weekly
14	Google Books	Niche book velocity	No key low volume, monthly
15	Common Crawl Index	Web-wide diffusion counts	No key, monthly

Tier A — L2 enrichment, top-K only

#	Source	Role	Budget
1	eBay Browse	Sold-vs-listed ratios	App key, 5,000 / day
2	GitHub REST	Star and fork velocity	PAT in 30 s, 5,000 / hr
3	Best Buy	Trending + Also-Viewed electronics	Instant key
4	Serper	Programmable SERP + PAA	2,500 free, no card
5	Product Hunt GraphQL	Launch velocity	Instant OAuth app
6	Walmart Affiliate	Mass-market trending items	Account + key upload
7	SearchAPI	Ad-library JSON sampling	100 free requests

Dropped for v1 automation (manual browser checks only)

- Reddit official API (access approval + commercial contract).
- Google Trends scraping (429 bans; official API is waitlisted).
- Keepa API (paid, no free tier) and Amazon hourly scraping (WAF + CAPTCHA).
- TikTok scraping (session wall, breaks often) and X API (\$200/mo plus legal risk).
- Meta Ad API (ID review, political/EU-only) and G2/Capterra bulk (anti-bot + ToS).
- Udemy API (dead for new developers) and Pinterest Trends (app review + OAuth upkeep).

End of Dossier 1. Dossier 2 (Builder Brief) tells the coding agent how to build all of the above, in which order, and what done looks like.